

Building a universal game-playing agent infrastructure to optimize performance

Maria Manina
IM updates

10.02.2026

Research question: reminder

Goal: create a general dialogue game-playing agent that

- does not rely on game-specific solvers
- does not modify game-specific rules
- must adhere to the standard clembench game flow

Method: adding different modular components around vanilla LLMs

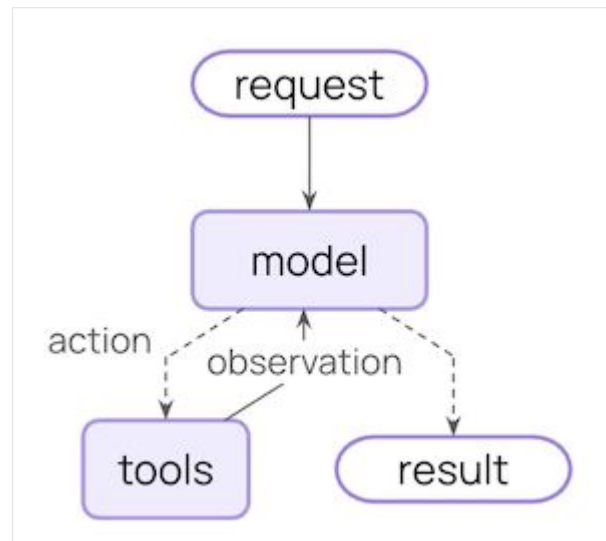
Previously:

- worked through the new clemcore environment architecture (agent-environment interface in PettingZoo and OpenEnv)
- implemented a minimal LangChain-based agent (verified end-to-end compatibility with the updated clemcore setup)

Agent architecture building (LangChain)

Where the variation happens in a LangChain agent:

- **system prompt** (tools available, desired behaviour, constraints, high-level instructions)
- **tools** (any number can be added; ReAct loop)
- **response format** (structured output)
- **additional components** (memory, evaluators)



Testing

- Chose 3 clembench games with diverse characteristics (single-player vs. multiplayer, word-based or exploration-based): **taboo**, **referencegame**, **textmapworld**
- Newly **generated instances** (v3.0), 15-25 instances per game
- Role testing: Agent vs. Human (fairer comparison, easier to identify errors), Agent vs. Agent

Agent architecture building (LangChain)

How I derived the agent components

- 1) Pragmatic needs (mostly from personal experience)
- 2) Papers on target domains and challenges of leveraging LLM capabilities in game playing agents

example source: A survey on **game playing agents** and large models: Methods, applications, and challenges [Xu et al. 2024]: Perception, Error Correction, Memory, Generalization

Not all individual agent components are necessary in every scenario

Baseline

A simple agent with short-term memory and a tag extraction subagent (gpt:oss-20b, t 0.0); prompted with one-shot examples

game	clemscore	% played	quality score	quality score (Std)
taboo	30.00	80.0	37.5	48.27
referencegame	79.17	95.83	82.61	38.76
textmapworld	56.79	88.0	64.53	16.16

Baseline

A taboo game example:

Step 4 / Reward 0.00:

```
Agent(player_0) <- Context: {'role': 'user', 'content': 'GUESS: support'}
```

```
Agent(player_0) -> Response: CLUE: Giving a boost to someone in need.
```

Step 5 / Reward 0.00:

```
Agent(player_1) <- Context: {'role': 'user', 'content': 'CLUE: Giving a boost to someone in need.'}
```

```
Agent(player_0) -> Response: I lose
```

Agent architecture building (LangChain)

Core tools (always on):

- observe (store important game flow information for later — visited placed, used guesses etc.) **Behaviour:consistency**
- recall (retrieve stored information)
- get_rules (stores the games rules and the output format) **Perception/(Error Correction?)**
- recall_rules (retrieve the stored information about the rules)

Optional tool sets (toggled per game based on auto-detection of keywords in the initial prompt) – ?

- gpt-4o-mini was used for testing since Ollama models performed poorly with tool use

Results

A gpt-4o-mini based agent with core tools (t 0.0)

game	clemscore	% played	quality score	quality score (Std)
taboo	34.53 	92.86 	37.18 	46.23
referencegame	54.17 	100.0 	54.17	50.9
textmapworld	0.0 	100.0 	0.0 	0.0

Results

- Agents sometimes failed to use tools despite explicit instructions
- Other cases showed excessive tool use or infinite ReAct loops
- However, there have also been cases where the tools were used properly
- Mitigation: enforced tool-usage limits in the agent prompt; max token and recursion limits occasionally caused early stopping
- Agent vs. Human testing revealed no discrepancies between the roles

Conclusion and further goals

- Run controlled ablations for tools
- Transition from exploratory notebook-based experimentation to a clemcore/clembench execution flow
- The multimodal game setting has not yet been explored